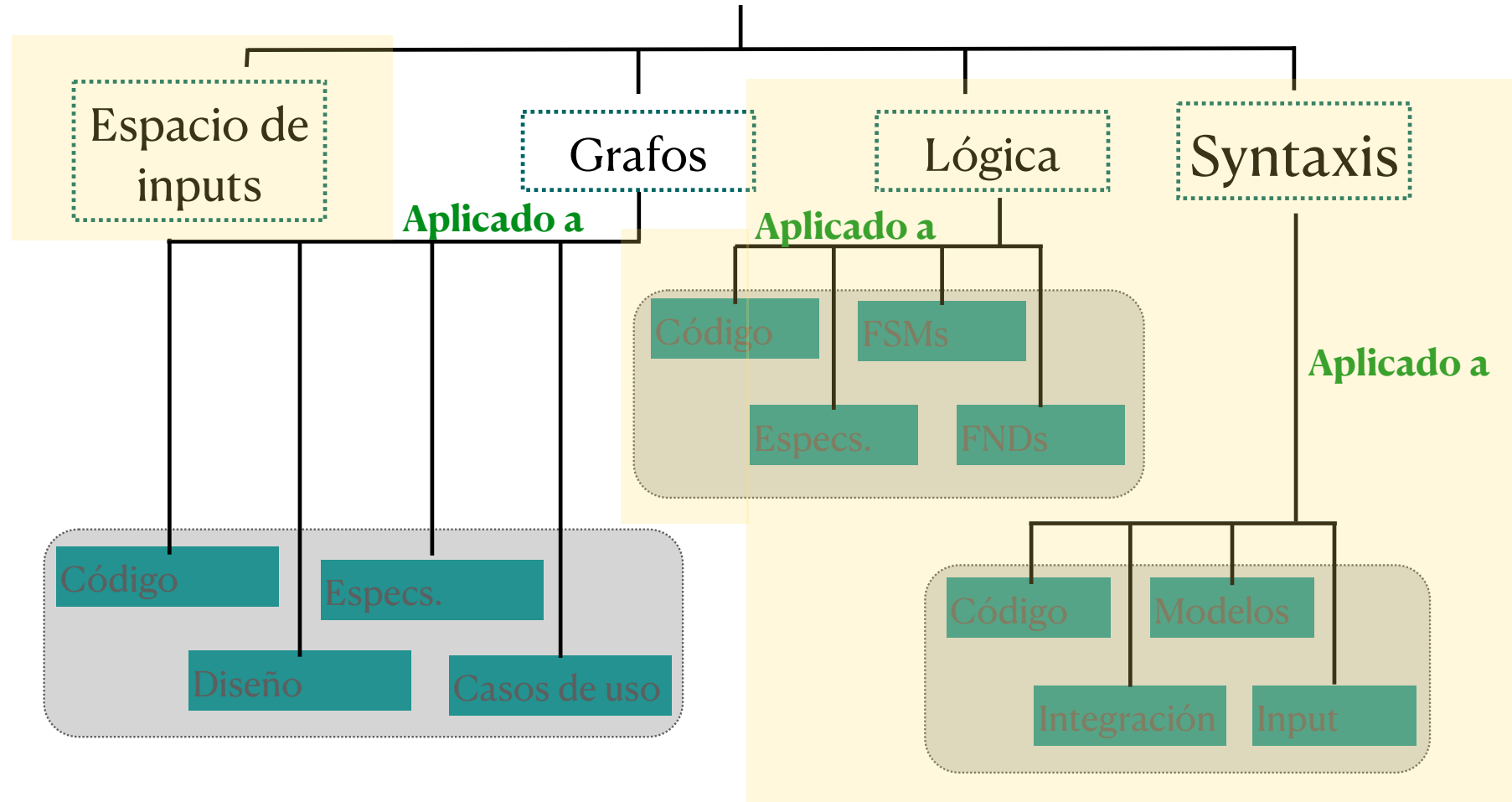


# Criterios cobertura de grafos: Introducción

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2<sup>nd</sup> Edition) y por Manuel Núñez (Versión en castellano)

# Cuatro estructuras para modelar software



# Recorriendo Grafos

Los grafos son la estructura **más utilizada** para realizar testing.

Los grafos pueden provenir de **muchos fuentes**.

- ★ Grafos para control de flujo.
- ★ Modelos de diseño.
- ★ FSMs y statecharts.
- ★ Casos de uso.

Los tests, usualmente, deben “cubrir” el grafo en un cierto sentido.

# Definición de un grafo

Un conjunto no vacío de **nod**os  $N$ .

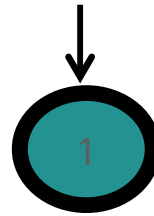
Un conjunto no vacío de **nod**os **iniciales**  $N_0$ .

Un conjunto no vacío de **nod**os **finales**  $N_f$ .

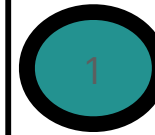
Un **conjunto de aristas**  $E$ . Cada aristas es un par de nodos  $(n_i, n_j)$ , donde  $n_i$  es el **predecesor** y  $n_j$  es el **sucesor**.

# Definición de un grafo

¿Es este un grafo?



## Referencias



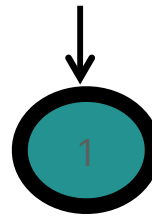
Estado final: doble círculo



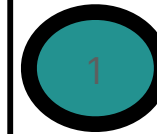
Estado Inicial

# Definición de un grafo

¿Es este un grafo?



## Referencias

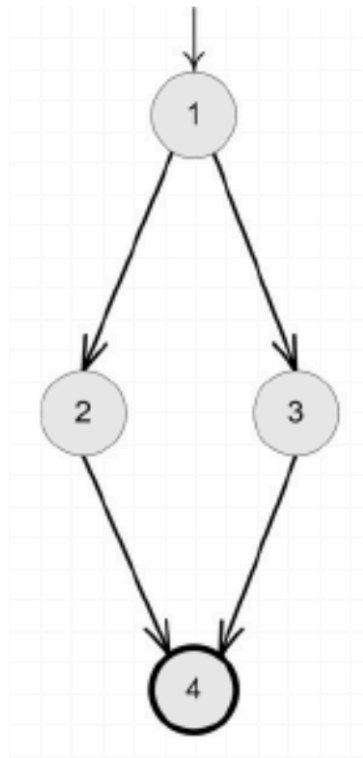


Estado final: doble círculo



Estado Inicial

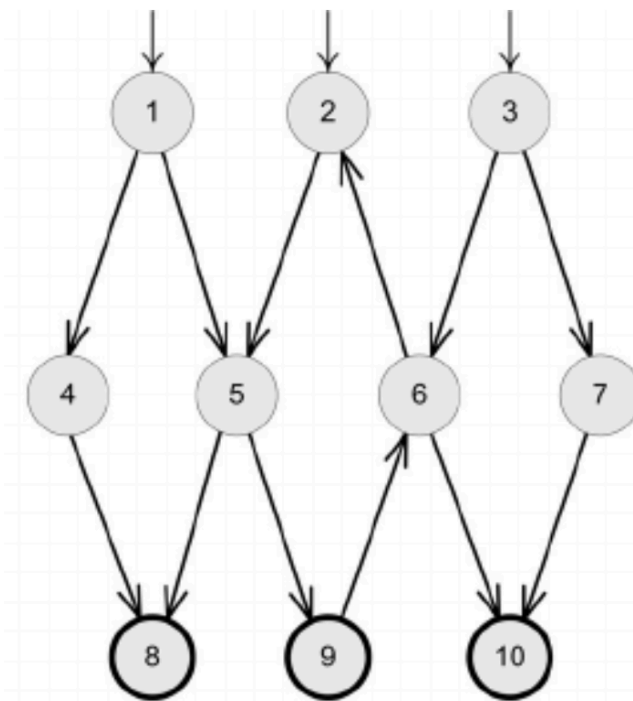
# Ejemplos de grafo



$$N_0 = \{1\}$$

$$N_f = \{4\}$$

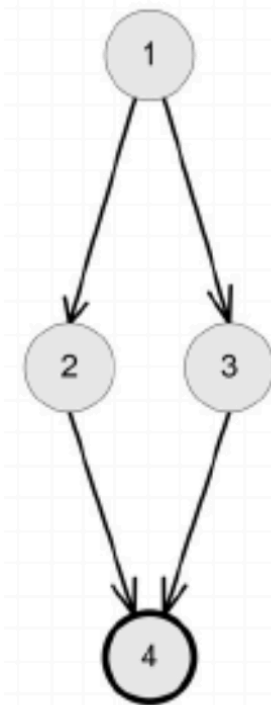
$$E = \{ (1,2), (1,3), (2,4), (3,4) \}$$



$$N_0 = \{1, 2, 3\}$$

$$N_f = \{8, 9, 10\}$$

$$E = \{ (1,4), (1,5), (2,5), (2,6), (3,6), (3,7), (4,8), (5,8), (5,9), (6,9), (6,10), (7,10), (9,6) \}$$

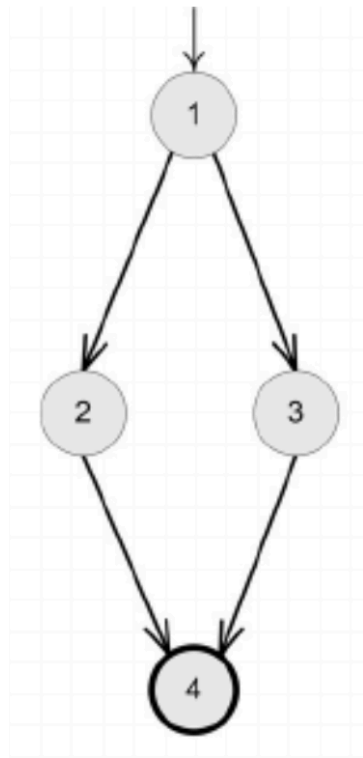


$$N_0 = \{ \}$$

$$N_f = \{4\}$$

$$E = \{ (1,2), (1,3), (2,4), (3,4) \}$$

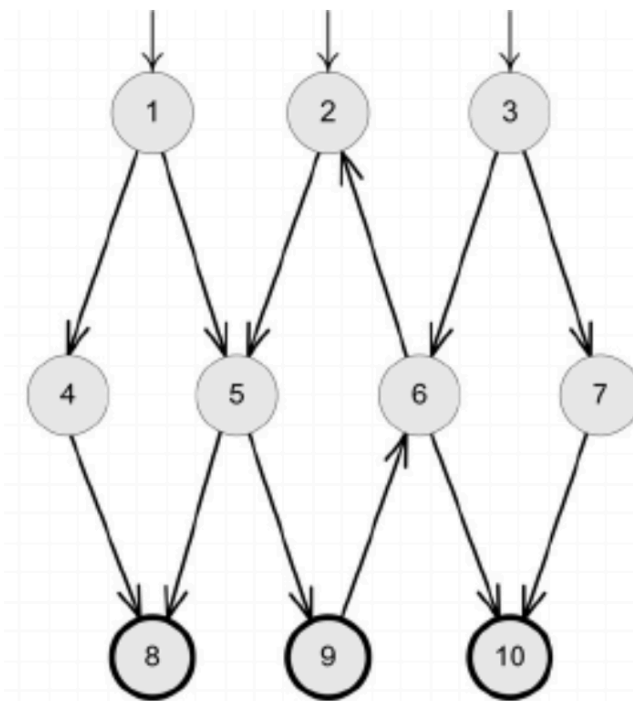
# Ejemplos de grafo



$$N_0 = \{1\}$$

$$N_f = \{4\}$$

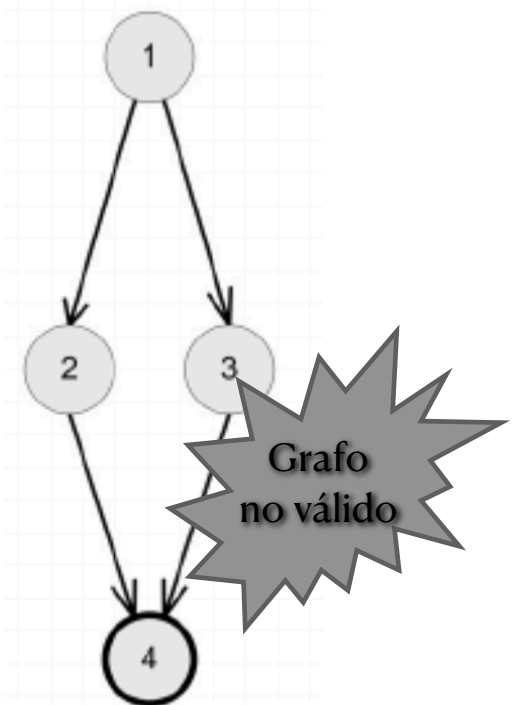
$$E = \{ (1,2), (1,3), (2,4), (3,4) \}$$



$$N_0 = \{1, 2, 3\}$$

$$N_f = \{8, 9, 10\}$$

$$E = \{ (1,4), (1,5), (2,5), (2,6), (3,6), (3,7), (4,8), (5,8), (5,9), (6,9), (6,10), (7,10), (9,6) \}$$



$$N_0 = \{ \}$$

$$N_f = \{4\}$$

$$E = \{ (1,2), (1,3), (2,4), (3,4) \}$$

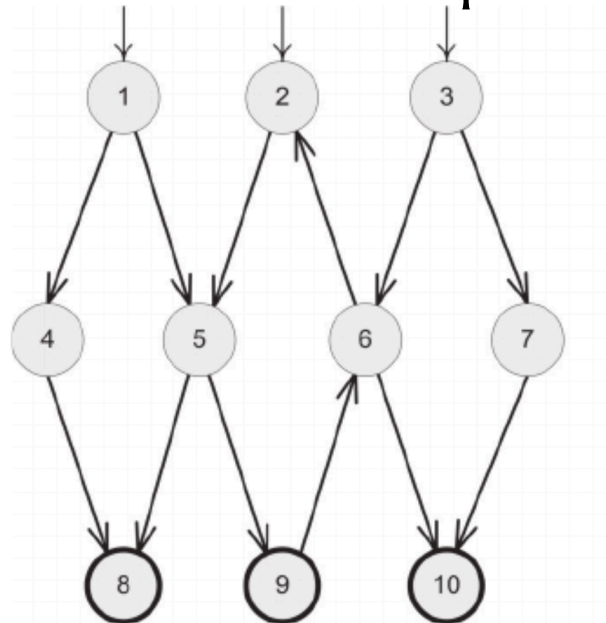


# Caminos en grafos

**Camino:** Secuencia de nodos  $[n_1, n_2, \dots, n_M]$  tal que cada par de nodos consecutivos esta conectado por una arista.

**Longitud:** Número de aristas.

**Subcamino:** Una subsecuencia de un camino  $p$  es un subcamino de  $p$

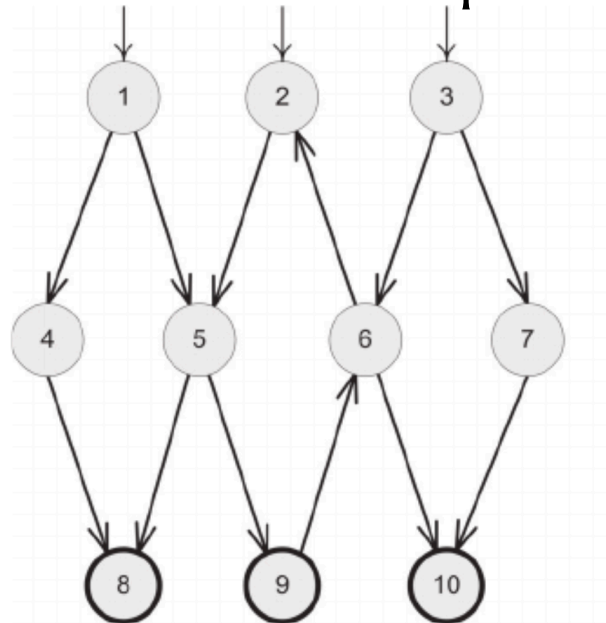


# Caminos en grafos

**Camino:** Secuencia de nodos  $[n_1, n_2, \dots, n_M]$  tal que cada par de nodos consecutivos esta conectado por una arista.

**Longitud:** Número de aristas.

**Subcamino:** Una subsecuencia de un camino  $p$  es un subcamino de  $p$



Algunos caminos

$[1, 4, 8]$

$[2, 5, 9, 6, 2]$

$[3, 7, 10]$

# Caminos de test y SESEs

**Camino de test:** Camino que empieza en un nodo inicial y termina en un nodo final.

Los caminos de test representan la ejecución de tests:

- ★ Algunos caminos de test pueden ser ejecutados por muchos tests.
- ★ Algunos caminos de test no se pueden ejecutar por ningún test.

Grafos **SESE** (Single Entry, Single Exit): Solo un nodo inicial y un nodo final.

# Camino de test y SESEs

Grafos **SESE** (Single Entry, Single Exit)

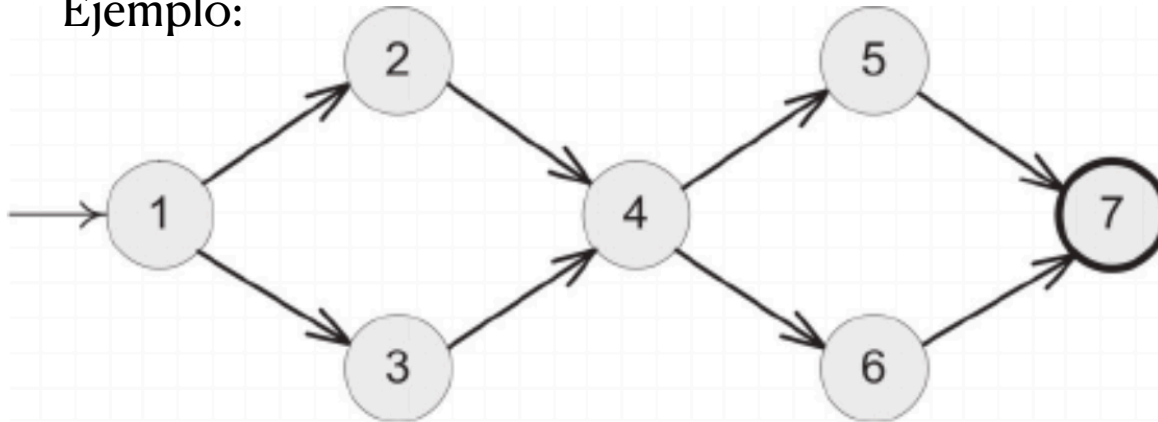
- ★ Solo un nodo inicial y un nodo final.
- ★  $N_o$  y  $N_f$  tienen exactamente un nodo
- ★ Todos los caminos de test comienzan en el mismo nodo y terminan en un mismo nodo

# Camino de test

- ★ Un nodo  $n$  es **sintácticamente** alcanzable desde el nodo  $n_i$  si existe un camino desde el nodo  $n_i$  hasta  $n$ .
- ★ Un nodo  $n$  también es **semánticamente** alcanzable si es posible ejecutar al menos un camino desde  $n_i$  hasta  $n$  con algún input.

# SESEs

Ejemplo:



En grafos SESEs:

- ★ el conjunto  $N_o$  tiene exactamente un nodo  $n_o$  y el conjunto  $N_f$  tiene exactamente un nodo  $n_f$ .
- ★  $n_f$  y  $n_o$  pueden ser el mismo nodo.
- ★  $n_f$  debe ser sintácticamente alcanzable desde todos los nodos en  $N$
- ★ Ningún nodo en  $N$  debe ser sintácticamente alcanzable desde  $n_f$ , salvo que  $n_f$  y  $n_o$  sean el mismo nodo.

# Visita y recorrido

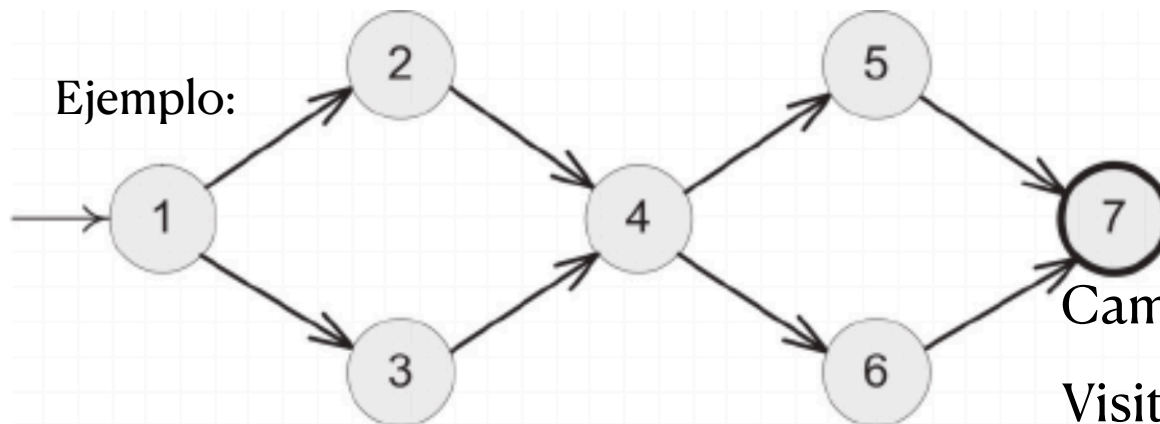
## Visita

- ★ Un camino de test  $p$  visita un nodo  $n$  si  $n$  está en  $p$ .
- ★ Un camino de test  $p$  visita una arista  $e$  si  $e$  está en  $p$ .

## Recorrido

- ★ Un camino de test  $p$  recorre un subcamino  $q$  si  $q$  es un subcamino de  $p$ .

# Visita y recorrido



Técnicamente cada  
arista también es un  
subcamino

Camino de test [ 1, 2, 4, 5, 7 ]

Visita nodos 1, 2, 4, 5, 7

Visita aristas (1, 2), (2, 4), (4, 5), (5, 7)

Recorre subcaminos

[1, 2, 4], [2, 4, 5], [4, 5, 7], [1, 2, 4, 5], [2, 4, 5, 7],  
[1, 2, 4, 5, 7],...



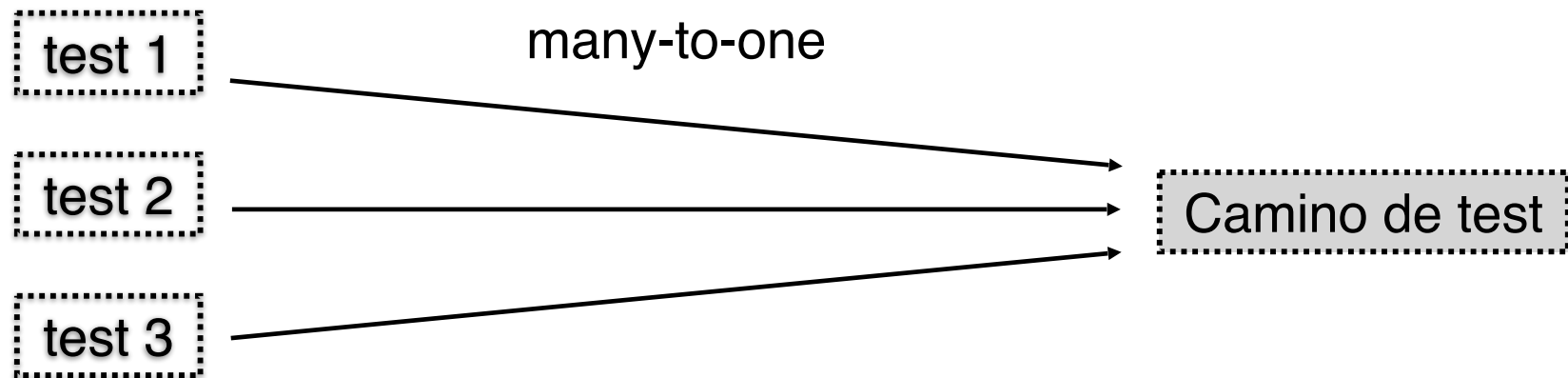
# Tests y caminos de tests

*camino(t)*: El camino de test ejecutado por el test  $t$ .

*caminos(T)*: El conjunto de caminos de test ejecutados por el conjunto de tests  $T$ .

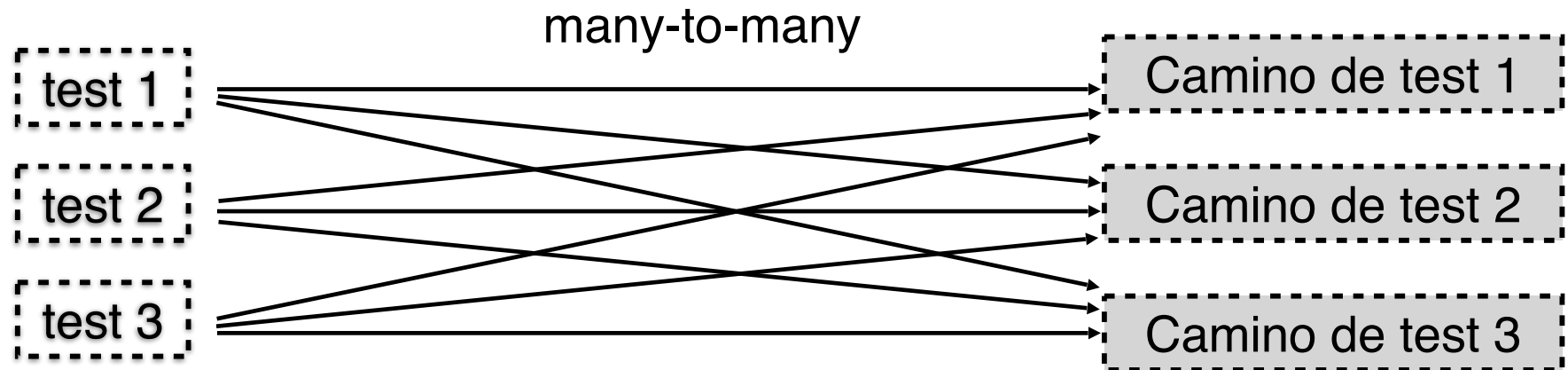
Cada test ejecuta solamente un camino de test (asumimos Programas determinísticos).

# Tests y caminos de tests



Software determinista: los test siempre ejecutan el mismo camino de test

# Tests y caminos de tests



Software no-determinista: El mismo test puede ejecutar distintos caminos de test

# Testing y cobertura de grafos

Utilizamos grafos en testing de la siguiente manera:

- ★ Desarrollamos un modelo del software en términos de un grafo.
- ★ Requerimos que los tests visiten/recorran conjuntos específicos de nodos, aristas y subcaminos.

**Requisitos de test:** Describen propiedades de los caminos de test.

**Criterio de test:** Reglas que definen los requisitos.

**Satisfacción:** Dado un conjunto de requisitos de test  $TR$  para un criterio  $C$ , un conjunto de tests  $T$  satisface  $C$  en un grafo sii para todo requisito  $tr$  del conjunto  $TR$ , existe un camino de test en  $\text{caminos}(T)$  que cumpla el requisito  $tr$ .

# Testing y cobertura de grafos

Podemos distinguir dos grandes grupos de criterios para cobertura de grafos.

# Testing y cobertura de grafos

Podemos distinguir dos grandes grupos de criterios para cobertura de grafos.

**Criterios de cobertura estructurales:** Se definen sobre un grafo en base a sus nodos y aristas.

# Testing y cobertura de grafos

Podemos distinguir dos grandes grupos de criterios para cobertura de grafos.

**Criterios de cobertura estructurales:** Se definen sobre un grafo en base a sus nodos y aristas.

**Criterios de cobertura sobre flujo de datos:** Requieren un grafo anotado con referencias a variables.

No lo vamos a  
ver en este  
asignatura.

# Criterios de cobertura estructurales

Veremos los siguientes criterios estructurales:

- ★ Cobertura de nodos



# Criterios de cobertura estructurales

Veremos los siguientes criterios estructurales:

- ★ Cobertura de nodos
- ★ Cobertura de aristas

# Cobertura de nodos

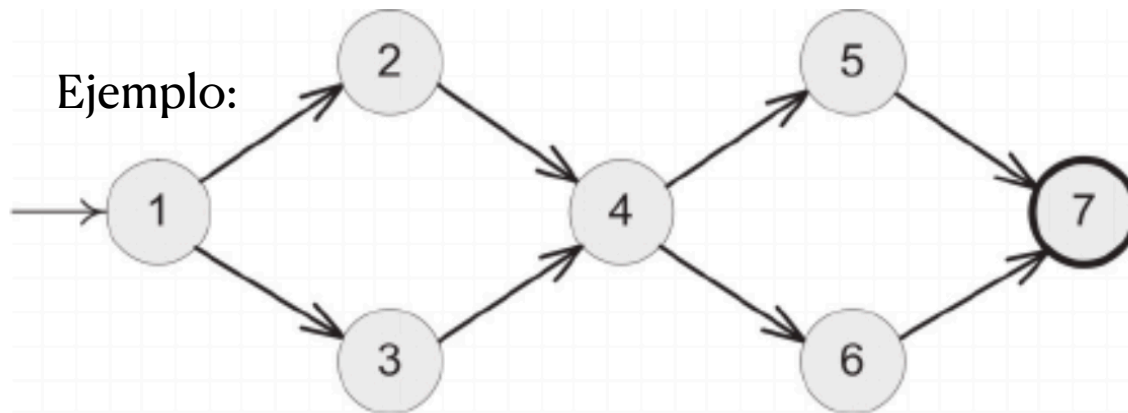
# Cobertura de nodos

**Node Coverage (NC):** El conjunto de tests  $T$  satisface NC en el grafo  $G$  sii para cada nodo sintácticamente alcanzable existe un camino  $p$  en  $\text{caminos}(T)$  tal que  $p$  lo visita.

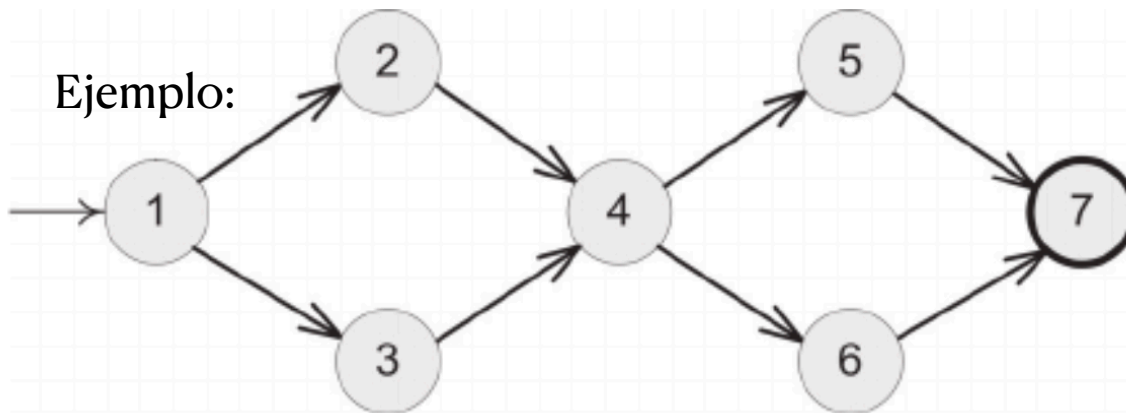
En términos de requisitos de test:

El conjunto de **requisitos de test**  $TR$  incluye todos los nodos alcanzables de  $G$

# Cobertura de nodos (NC)



# Cobertura de nodos (NC)



$$TR = \{1, 2, 3, 4, 5, 6, 7\}$$

Caminos de test para obtener cobertura de nodos:

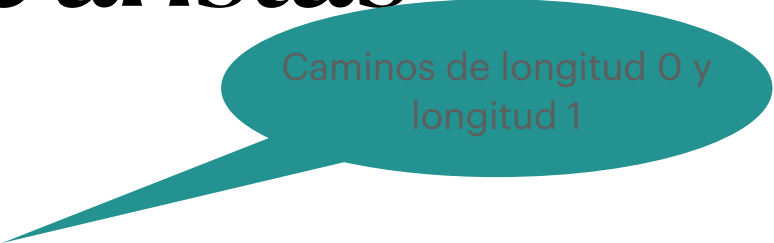
$$p_1 = [1, 2, 4, 5, 7] \text{ y } p_2 = [1, 3, 4, 6, 7]$$

Por lo tanto, si un conjunto de  $T$  contiene  $\{t_1, t_2\}$ , donde **camino**( $t_1$ ) =  $p_1$  and **camino**( $t_2$ ) =  $p_2$ , entonces  $T$  satisface **NC** on  $G$ .

# Cobertura de aristas

**Edge Coverage (EC):** El conjunto de **requisitos de test**  $TR$  incluye cada camino de longitud a lo sumo 1 incluido en  $G$ .

# Cobertura de aristas



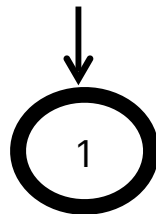
Caminos de longitud 0 y  
longitud 1

**Edge Coverage (EC):** El conjunto de **requisitos de test**  $TR$  incluye cada camino de longitud a lo sumo 1 incluido en  $G$ .

# Cobertura de aristas

Caminos de longitud 0 y longitud 1

**Edge Coverage (EC):** El conjunto de **requisitos de test**  $TR$  incluye cada camino de longitud a lo sumo 1 incluido en  $G$ .



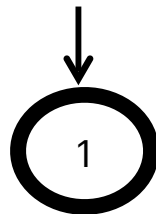
Debo cubrir el nodo!  
Camino de longitud cero deben ser cubiertos!



# Cobertura de aristas

Caminos de longitud 0 y longitud 1

**Edge Coverage (EC)**: El conjunto de **requisitos de test**  $TR$  incluye cada camino de longitud a lo sumo 1 incluido en  $G$ .

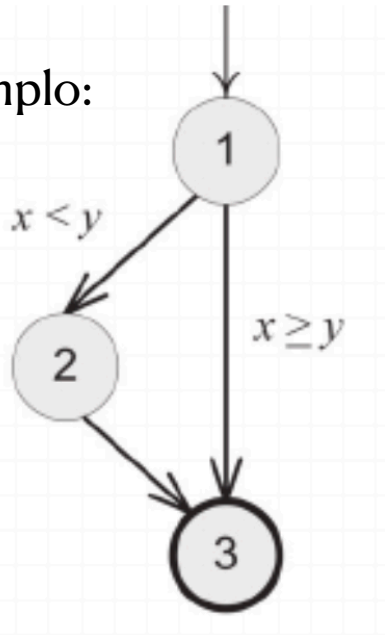


Debo cubrir el nodo!  
Camino de longitud cero deben ser cubiertos!

**Edge Coverage** subsume a **Node Coverage**

# Cobertura de Nodos y Aristas

Ejemplo:



NC:  $RT = \{ 1, 2, 3 \}$

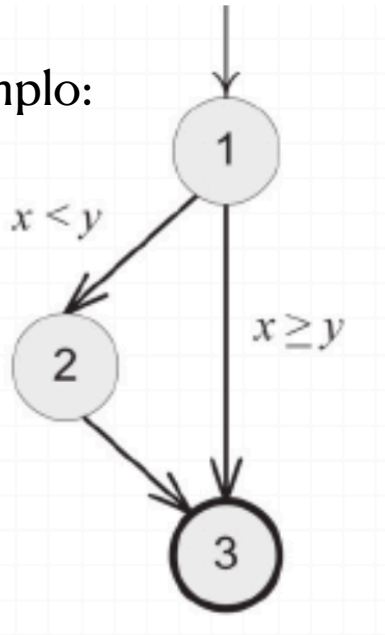
Camino de test =  $[ 1, 2, 3 ]$

EC:  $RT = \{ (1, 2), (1, 3), (2, 3) \}$

Caminos de tests =  $[ 1, 2, 3 ] [ 1, 3 ]$

# Cobertura de Nodos y Aristas

Ejemplo:



NC y EC difieren solamente cuando existen dos nodos que están unidos por una arista y por otro subcamino (típicamente, esto ocurre en instrucciones “if sin else”).

NC:  $RT = \{ 1, 2, 3 \}$

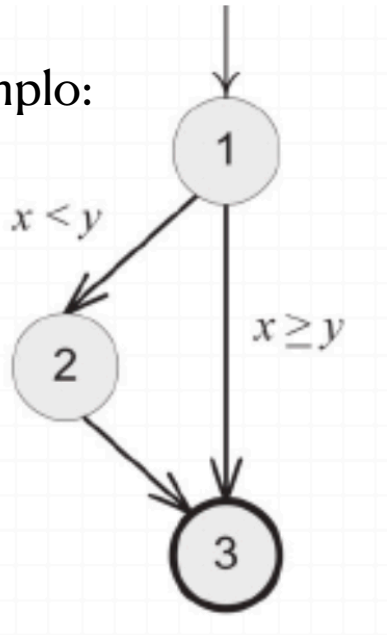
Camino de test =  $[ 1, 2, 3 ]$

EC:  $RT = \{ (1, 2), (1, 3), (2, 3) \}$

Caminos de tests =  $[ 1, 2, 3 ] [ 1, 3 ]$

# Cobertura de Nodos y Aristas

Ejemplo:



NC:  $RT = \{ 1, 2, 3 \}$

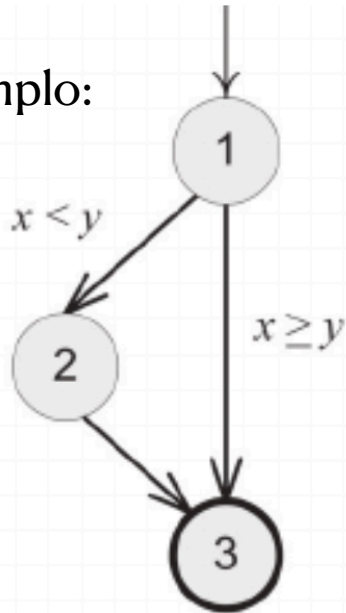
Camino de test =  $[ 1, 2, 3 ]$

EC:  $RT = \{ (1, 2), (1, 3), (2, 3) \}$

Caminos de tests =  $[ 1, 2, 3 ] [ 1, 3 ]$

# Cobertura de Nodos y Aristas

Ejemplo:



La cobertura de nodo se implementa en muchas herramientas en forma de cobertura de sentencias. Cobertura de aristas, generalmente implementado como cobertura de branches.

NC:  $RT = \{ 1, 2, 3 \}$

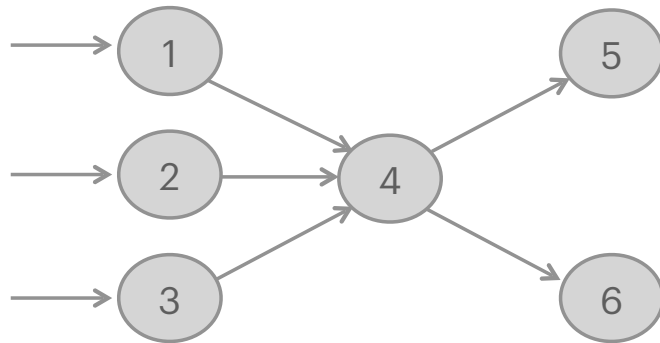
Camino de test =  $[ 1, 2, 3 ]$

EC:  $RT = \{ (1, 2), (1, 3), (2, 3) \}$

Camino de tests =  $[ 1, 2, 3 ] [ 1, 3 ]$

# Cobertura de Pares de Aristas

**Edge-Pair Coverage (EPC):** RT incluye cada camino de longitud a lo sumo 2 incluido en  $G$ . La cláusula “a lo sumo 2” permite grafos con menos de dos aristas.

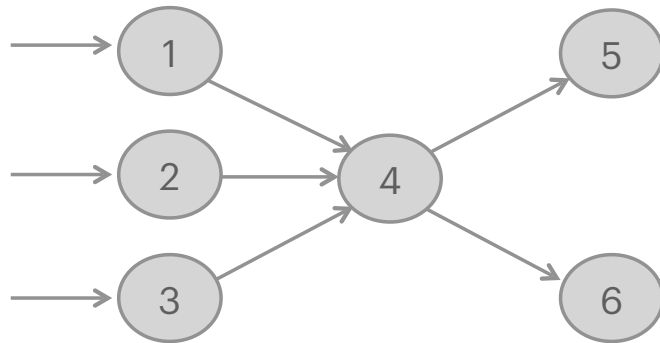


EPC:

$RT = \{ [1,4,5], [1,4,6], [2,4,5], [2,4,6], [3,4,5], [3,4,6] \cup RT_{EC} \}$

# Cobertura de Pares de Aristas

**Edge-Pair Coverage (EPC):** RT incluye cada camino de longitud a lo sumo 2 incluido en  $G$ . La cláusula “a lo sumo 2” permite grafos con menos de dos aristas.



EPC:

$RT = \{ [1,4,5], [1,4,6], [2,4,5], [2,4,6], [3,4,5], [3,4,6] \cup RT_{EC} \}$

La extensión lógica es requerir todos los caminos ...

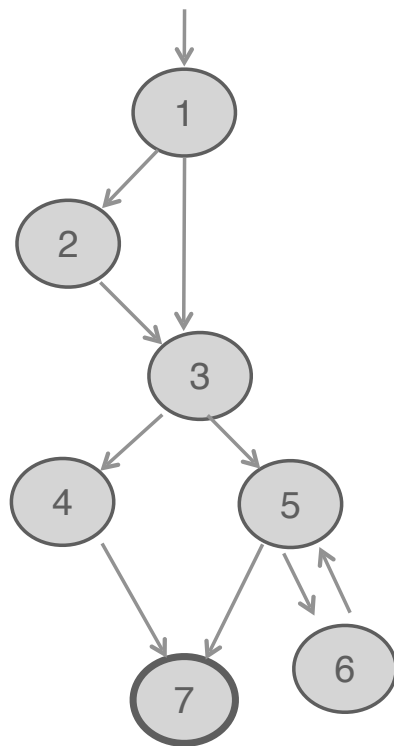
# Complete Path Coverage

**Complete Path Coverage (CPC):** RT incluye todos los caminos de  $G$ .

★ Este criterio es imposible de llevar a cabo si el grafo tiene un bucle.



# Cobertura Estructural: Ejemplo



## Node Coverage

$RT = \{1, 2, 3, 4, 5, 6, 7\}$ , Caminos de test:  $[1, 2, 3, 4, 7]$   $[1, 2, 3, 5, 6, 5, 7]$

## Edge Coverage

$RT = \{(1,2), (1,3), (2,3), (3,4), (3,5), (4,7), (5,6), (5,7), (6,5)\}$ ,

Caminos de test :  $[1, 2, 3, 4, 7]$   $[1, 3, 5, 6, 5, 7]$

## Edge-Pair Coverage

$RT = \{[1,2,3], [1,3,4], [1,3,5], [2,3,4], [2,3,5], [3,4,7], [3,5,6], [3,5,7], [5,6,5], [6,5,6], [6,5,7]\}$

Caminos de test :  $[1, 2, 3, 4, 7]$   $[1, 2, 3, 5, 7]$   $[1, 3, 4, 7]$   $[1, 3, 5, 6, 5, 6, 5, 7]$

## Complete Path Coverage

Caminos de test :  $[1, 2, 3, 4, 7]$   $[1, 2, 3, 5, 7]$   $[1, 2, 3, 5, 6, 5, 6]$

$[1, 2, 3, 5, 6, 5, 6, 5, 7]$   $[1, 2, 3, 5, 6, 5, 6, 5, 6, 5, 7] \dots$

# Camino simple y principales

Si un grafo tiene un bucle entonces tiene un número **infinito** de caminos. Por tanto, CPC **no** se puede **realizar**.

Nuevas Definiciones.

- ★ **Camino simple**: Un camino donde no se repiten nodos (salvo, posiblemente, el último). Por ejemplo, una vuelta de bucle es un camino simple (pero no se permiten bucles internos).
- ★ Un camino de  $n_i$  a  $n_j$  es simple si ningún nodo aparece más de una vez en el camino, con la excepción del primer y el último nodo.
- ★ Los caminos simples no tienen bucles internos, aunque el camino completo puede ser un bucle.

# Caminos Simples y Principales

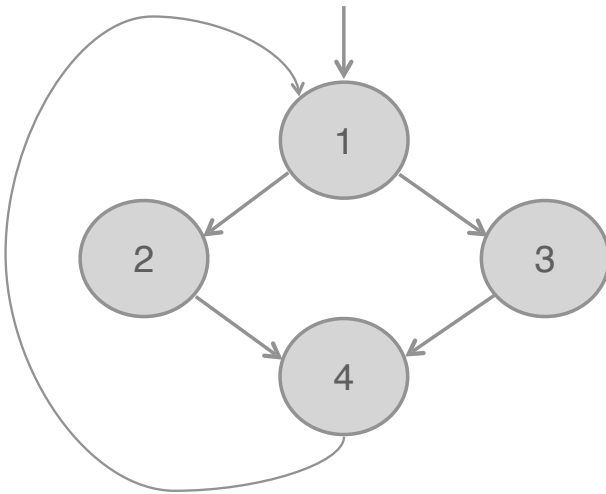
Si un grafo tiene un bucle entonces tiene un número **infinito** de caminos. Por tanto, CPC **no** se puede **realizar**.

Nuevas Definiciones.

En el contexto de un criterio de Cobertura **nos interesan caminos simples de longitud maximal.**

**Camino Principal** (*prime path*): Un camino de  $n_i$  a  $n_j$  es un camino principal si es un camino simple y no aparece como un subcamino propio de ningún otro camino simple.

# Camino Simple y Principales



Camino simple : [1,2,4,1], [1,3,4,1], [2,4,1,2], [2,4,1,3], [3,4,1,2], [3,4,1,3], [4,1,2,4], [4,1,3,4], [1,2,4], [1,3,4], [2,4,1], [3,4,1], [4,1,2], [4,1,3], [1,2], [1,3], [2,4], [3,4], [4,1], [1], [2], [3], [4]

Camino principal : [2,4,1,2], [2,4,1,3], [1,3,4,1], [1,2,4,1], [3,4,1,2], [4,1,3,4], [4,1,2,4], [3,4,1,3]

# Cobertura de caminos principales

**Prime Path Coverage (PPC):** RT incluye todos los caminos principales de  $G$ .

Un criterio simple, elegante y finito que requiere tanto que los bucles se ejecuten como que se salten.

Recorrerá todos los caminos de longitud 0, 1, ...

Por tanto, subsume cobertura de nodos y de aristas.

PPC casi, pero no completamente, subsume EPC...

**Satisfacción:** Dado un conjunto de requisitos de test  $TR$  para un criterio  $C$ , un conjunto de tests  $T$  satisface  $C$  en un grafo sii para todo requisito  $tr$  del conjunto  $TR$ , existe un camino de test en  $\text{caminos}(T)$  que **recorra** el requisito  $tr$ .

# Cobertura de caminos principales

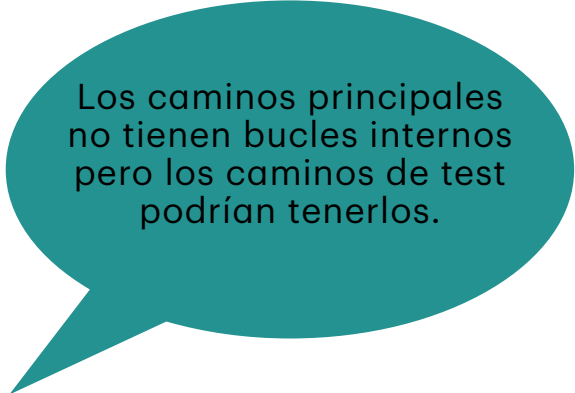
**Prime Path Coverage (PPC):** RT incluye todos los caminos principales de  $G$ .

Un criterio simple, elegante y finito que requiere tanto que los bucles se ejecuten como que se salten.

Recorrerá todos los caminos de longitud 0, 1, ...

Por tanto, subsume cobertura de nodos y de aristas.

PPC casi, pero no completamente, subsume EPC...



Los caminos principales no tienen bucles internos pero los caminos de test podrían tenerlos.

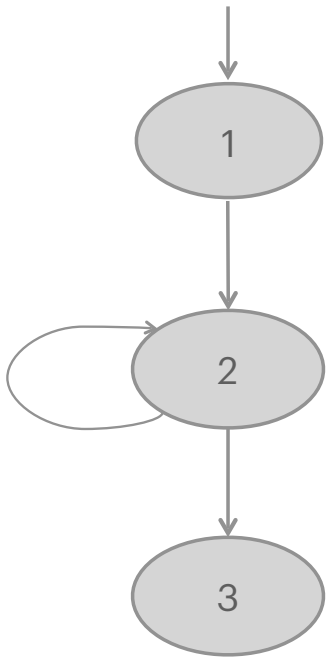
**Satisfacción:** Dado un conjunto de requisitos de test  $TR$  para un criterio  $C$ , un conjunto de tests  $T$  satisface  $C$  en un grafo sii para todo requisito  $tr$  del conjunto  $TR$ , existe un camino de test en  $\text{caminos}(T)$  que **recorra** el requisito  $tr$ .

# PPC casi, pero no subsume EPC...

Si un nodo  $n$  tiene una arista hacia si mismo (bucle), EPC requiere  $[n, n, m]$  y  $[m, n, n]$  pero ninguno de los dos es simple (y por tanto no es primario).

# PPC casi, pero no subsume EPC...

Si un nodo  $n$  tiene una arista hacia si mismo (bucle), EPC requiere  $[n, n, m]$  y  $[m, n, n]$  pero ninguno de los dos es simple (y por tanto no es primario).



Requisitos de Test EPC:  $\{ [1,2,3], [1,2,2], [2,2,3], [2,2,2] \}$

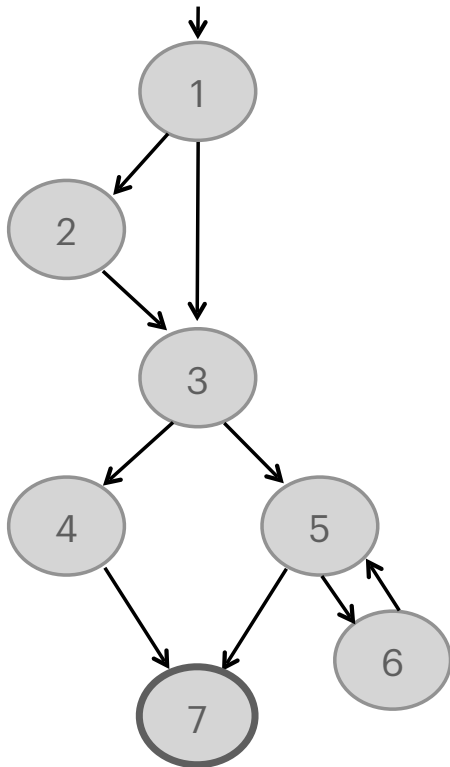
Requisitos de Test PPC:  $\{ [1,2,3], [2,2] \}$



# Caminos Principales: ejemplo

Caminos Simples?

**Caminos Principales?**



Referencias

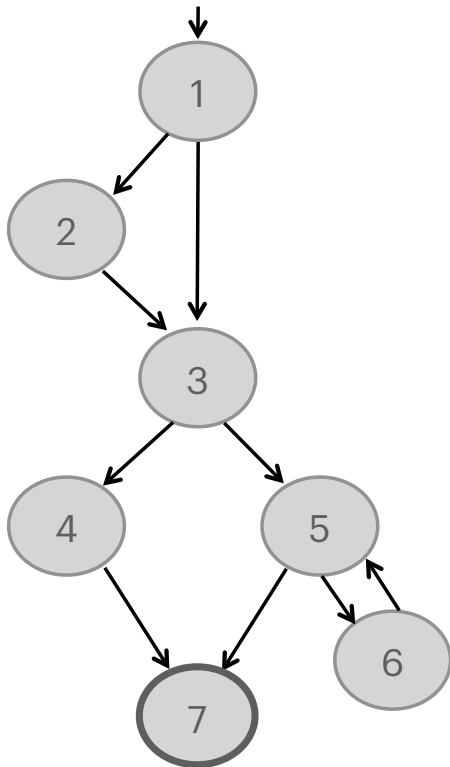
! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo

Caminos Simples?

**Caminos Principales?**



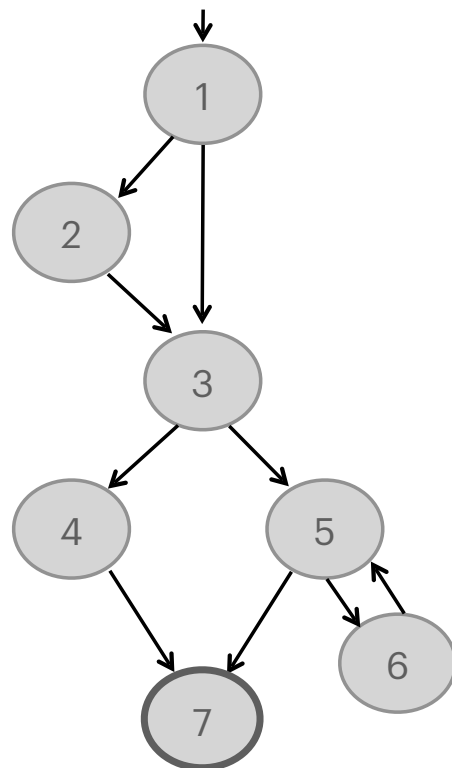
Long o  
[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo



Caminos Simples?

Long 0

[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Long 1

[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

**Caminos Principales?**

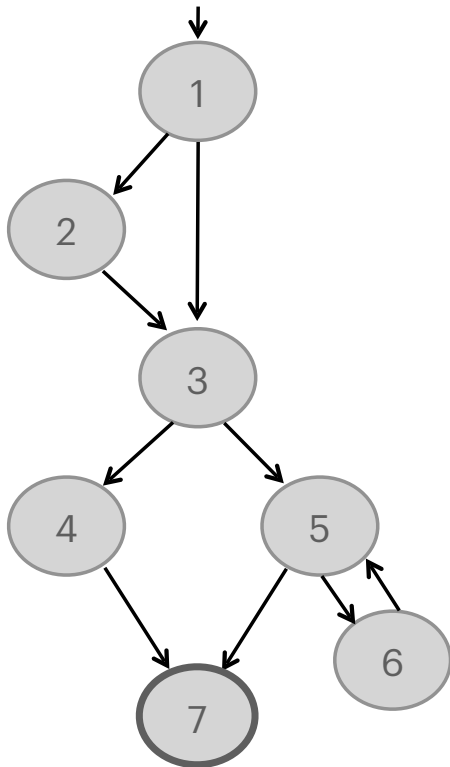
Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo

## Caminos Principales?



## Caminos Simples?

### Long 0

[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

### Long 1

[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

### Long 2

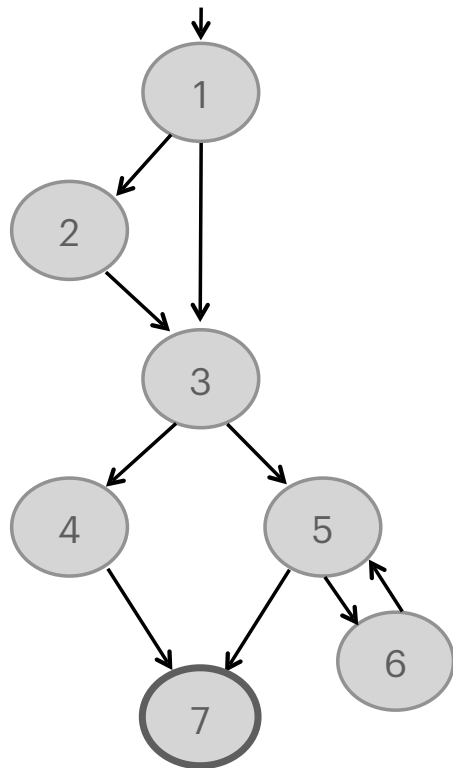
[1, 2, 3]  
[1, 3, 4]  
[1, 3, 5]  
[2, 3, 4]  
[2, 3, 5]  
[3, 4, 7]!  
[3, 5, 7]!  
[3, 5, 6]!  
[5, 6, 5] \*  
[6, 5, 7]!  
[6, 5, 6] \*

## Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo



Caminos Simples?

Long 0  
[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Long 1  
[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

Long 2  
[1, 2, 3]  
[1, 3, 4]  
[1, 3, 5]  
[2, 3, 4]  
[2, 3, 5]  
[3, 4, 7]!  
[3, 5, 7]!  
[3, 5, 6]!  
[5, 6, 5] \*  
[6, 5, 7]!  
[6, 5, 6] \*

**Caminos Principales?**

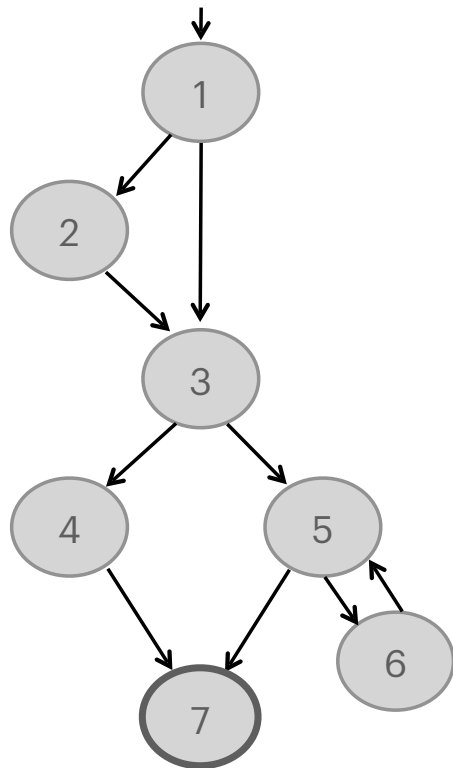
Long 3  
[1, 2, 3, 4]  
[1, 2, 3, 5]  
[1, 3, 4, 7]!  
[1, 3, 5, 7]!  
[1, 3, 5, 6]!  
[2, 3, 4, 7]!  
[2, 3, 5, 6]!  
[2, 3, 5, 7]!

Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo



## Caminos Simples?

Long 0  
[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Long 1  
[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

Long 2  
[1, 2, 3]  
[1, 3, 4]  
[1, 3, 5]  
[2, 3, 4]  
[2, 3, 5]  
[3, 4, 7]!  
[3, 5, 7]!  
[3, 5, 6]!  
[5, 6, 5] \*  
[6, 5, 7]!  
[6, 5, 6] \*

## Caminos Principales?

Long 3  
[1, 2, 3, 4]  
[1, 2, 3, 5]  
[1, 3, 4, 7]!  
[1, 3, 5, 7]!  
[1, 3, 5, 6]!  
[2, 3, 4, 7]!  
[2, 3, 5, 6]!  
[2, 3, 5, 7]!

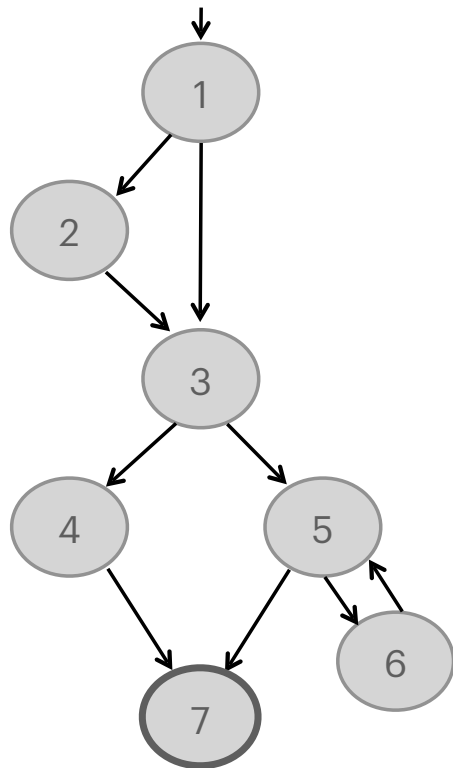
Lon 4  
[1, 2, 3, 4, 7]!  
[1, 2, 3, 5, 7]!  
[1, 2, 3, 5, 6]!

## Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo



## Caminos Simples?

Long 0  
[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Long 1  
[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

Long 2  
[1, 2, 3]  
[1, 3, 4]  
[1, 3, 5]  
[2, 3, 4]  
[2, 3, 5]  
[3, 4, 7]!  
[3, 5, 7]!  
[3, 5, 6]!  
**[5, 6, 5] \***  
**[6, 5, 7] !**  
**[6, 5, 6] \***

## Caminos Principales?

Long 3  
[1, 2, 3, 4]  
[1, 2, 3, 5]  
[1, 3, 4, 7]!  
[1, 3, 5, 7]!  
[1, 3, 5, 6]!  
[2, 3, 4, 7]!  
[2, 3, 5, 6]!  
[2, 3, 5, 7]!

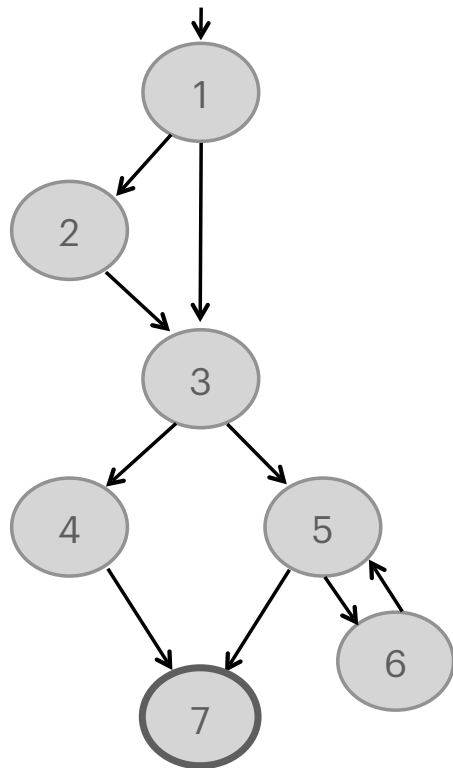
Lon 4  
[1, 2, 3, 4, 7]!  
[1, 2, 3, 5, 7]!  
[1, 2, 3, 5, 6]!

## Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo



## Caminos Simples?

Long 0  
[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Long 1  
[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

Long 2  
[1, 2, 3]  
[1, 3, 4]  
[1, 3, 5]  
[2, 3, 4]  
[2, 3, 5]  
[3, 4, 7]!  
[3, 5, 7]!  
[3, 5, 6]!  
[5, 6, 5] \*  
[6, 5, 7]!  
[6, 5, 6] \*

## Caminos Principales?

Long 3  
[1, 2, 3, 4]  
[1, 2, 3, 5]  
**[1, 3, 4, 7]!**  
**[1, 3, 5, 7]!**  
**[1, 3, 5, 6]!**  
[2, 3, 4, 7]!  
[2, 3, 5, 6]!  
[2, 3, 5, 7]!

Lon 4  
[1, 2, 3, 4, 7]!  
[1, 2, 3, 5, 7]!  
[1, 2, 3, 5, 6]!

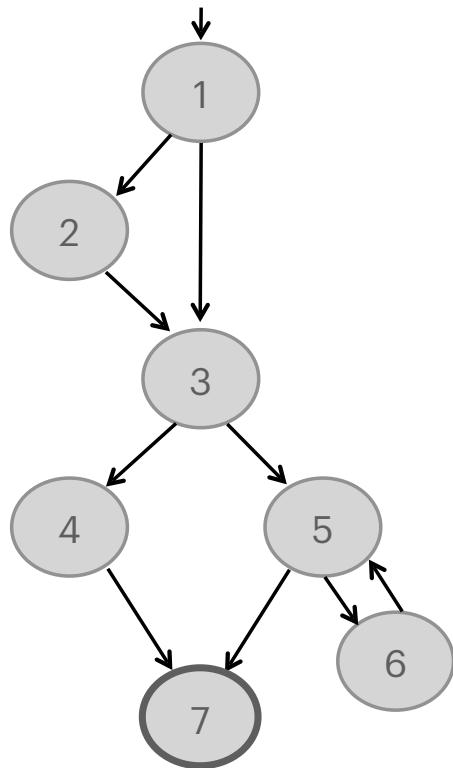
## Referencias

! Caminos que no se pueden extender

\*caminos cíclicos



# Caminos Principales: ejemplo



## Caminos Simples?

Long 0  
[1]  
[2]  
[3]  
[4]  
[5]  
[6]  
[7]!

Long 1  
[1, 2]  
[1, 3]  
[2, 3]  
[3, 4]  
[3, 5]  
[4, 7]!  
[5, 7]!  
[5, 6]  
[6, 5]

Long 2  
[1, 2, 3]  
[1, 3, 4]  
[1, 3, 5]  
[2, 3, 4]  
[2, 3, 5]  
[3, 4, 7]!  
[3, 5, 7]!  
[3, 5, 6]!  
[5, 6, 5] \*  
[6, 5, 7]!  
[6, 5, 6] \*

## Caminos Principales?

Long 3  
[1, 2, 3, 4]  
[1, 2, 3, 5]  
[1, 3, 4, 7]!  
[1, 3, 5, 7]!  
[1, 3, 5, 6]!  
[2, 3, 4, 7]!  
[2, 3, 5, 6]!  
[2, 3, 5, 7]!

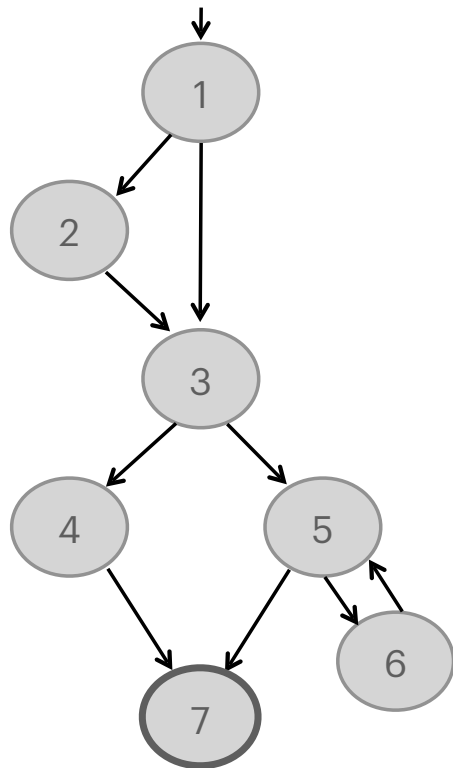
Lon 4  
[1, 2, 3, 4, 7]!  
[1, 2, 3, 5, 7]!  
[1, 2, 3, 5, 6]!

## Referencias

! Caminos que no se pueden extender

\*caminos cíclicos

# Caminos Principales: ejemplo



## Caminos Principales

[1, 2, 3, 4, 7]

[1, 2, 3, 5, 7]

[1, 2, 3, 5, 6]

[1, 3, 4, 7]

[1, 3, 5, 7] Ejecutan el ciclo 0 veces

[1, 3, 5, 6] Ejecutan el ciclo 1 vez

[5, 6, 5]

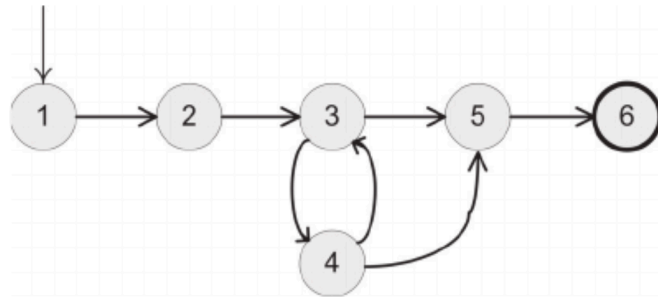
[6, 5, 7]

[6, 5, 6] Ejecutan el ciclo más de una vez

# Requisitos de Test Inviabiles

Hay **requisitos de test** que **no** se pueden **satisfacer**. Por ejemplo:

Código muerto.



Si el loop en este grafo puede ser ejecutado cero veces entonces el camino [1,2,3,5,6] puede ser satisfecho sino no.

La mayoría de los **criterios** de test cuenta con **requisitos** de test **inviabiles**.

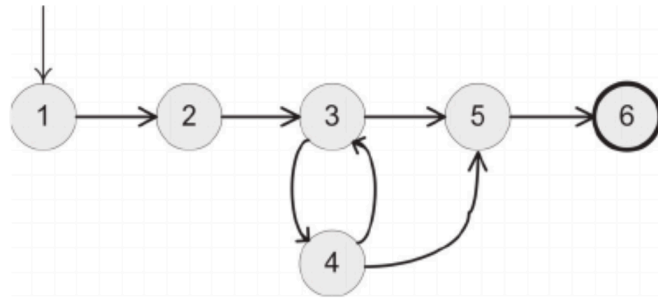
Usualmente es **indecidable** saber si los requisitos son posibles.

# Requisitos de Test Inviabiles

Hay **requisitos de test** que **no** se pueden **satisfacer**. Por ejemplo:

Código muerto.

Subcamino que solo se ejecuta con una contradicción ( $x > 0 \ \&\& \ x < 0$ ).



Si el loop en este grafo puede ser ejecutado cero veces entonces el camino [1,2,3,5,6] puede ser satisfecho sino no.

La mayoría de los **criterios** de test cuenta con **requisitos** de test **inviabiles**.

Usualmente es **indecidable** saber si los requisitos son posibles.

# Tours, sidetrips, detours

Los caminos primarios no tienen bucles internos pero los caminos de test podrían tenerlos.

**Tour:** Un camino de test  $p$  **recorre** (tours) un subcamino  $q$  si  $q$  es un subcamino de  $p$ .

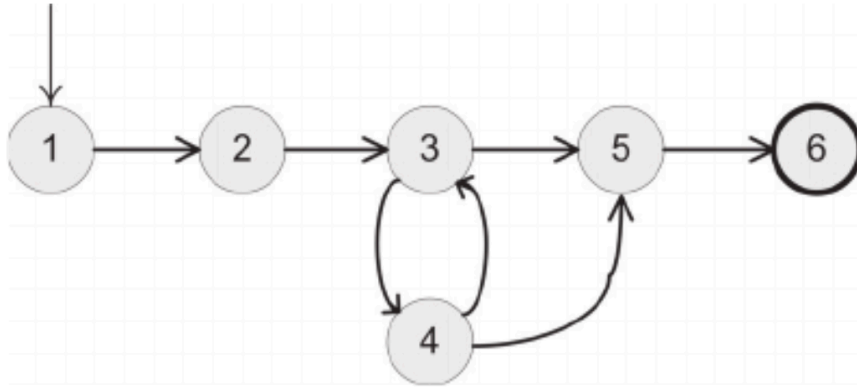
**Tour con *sidetrips* (viaje adicional):** Un camino de test  $p$  recorre un subcamino  $q$  con viajes adicionales sii cada arista de  $q$  está también en  $p$  y en el mismo orden.

Un *tour* puede incluir un ***sidetrip*** siempre que se retorne al mismo nodo.

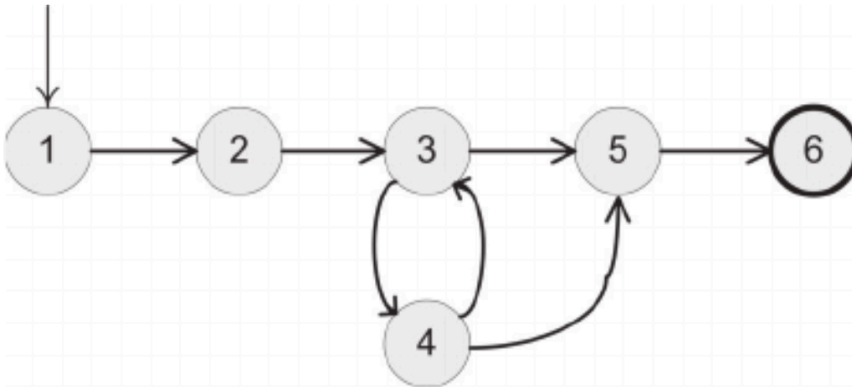
**Tour con *detours* (desvíos):** Un camino de test  $p$  recorre un subcamino  $q$  con desvíos sii cada nodo de  $q$  está también en  $p$  y en el mismo orden.

Un *tour* puede incluir un ***detour*** a partir de un nodo siempre que se vuelva al camino principal de un sucesor de dicho nodo.

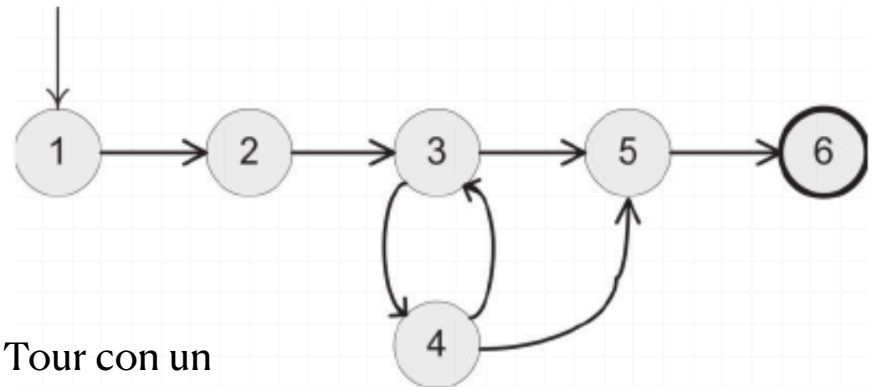
# Ejemplo



Tour del camino  
primario [1, 2, 3, 5, 6]  
sin sidetrips ni detours

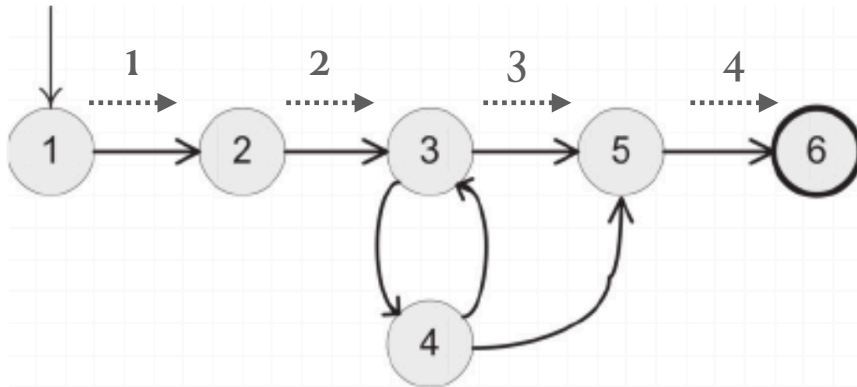


Tour con un  
sidetrip

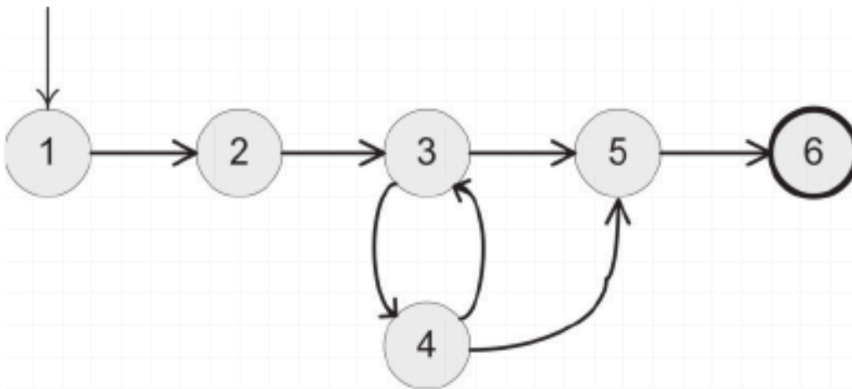


Tour con un  
detour

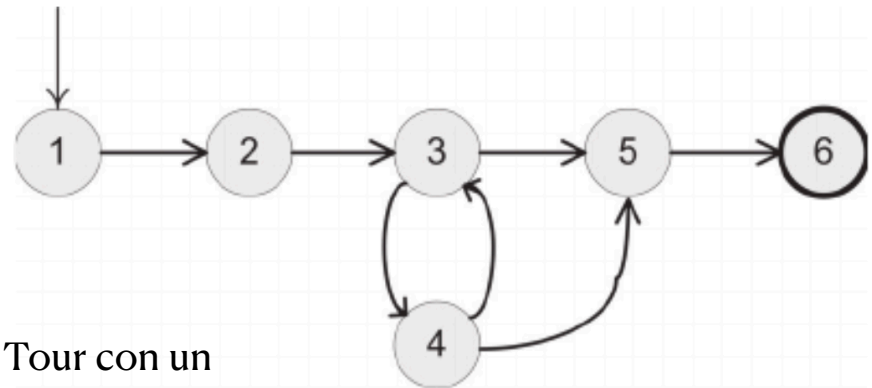
# Ejemplo



Tour del camino  
primario [1, 2, 3, 5, 6]  
sin sidetrips ni detours

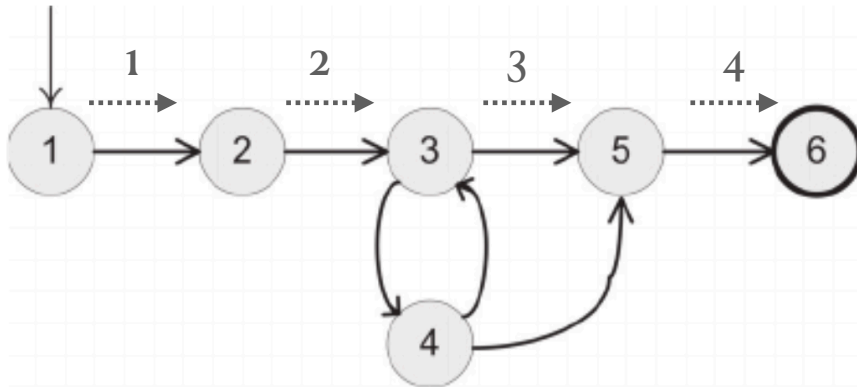


Tour con un  
sidetrip

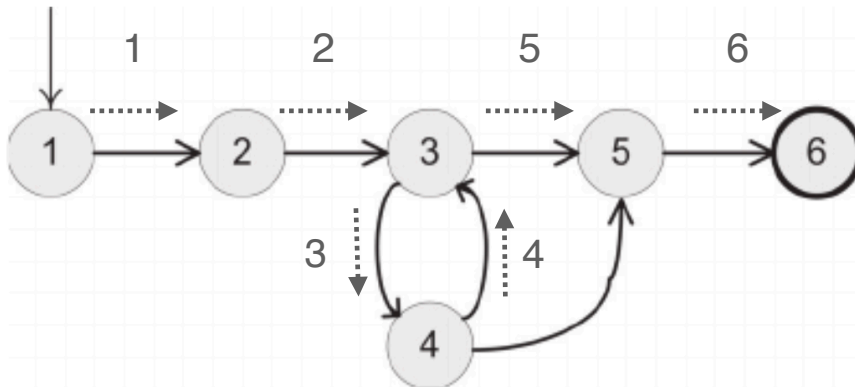


Tour con un  
detour

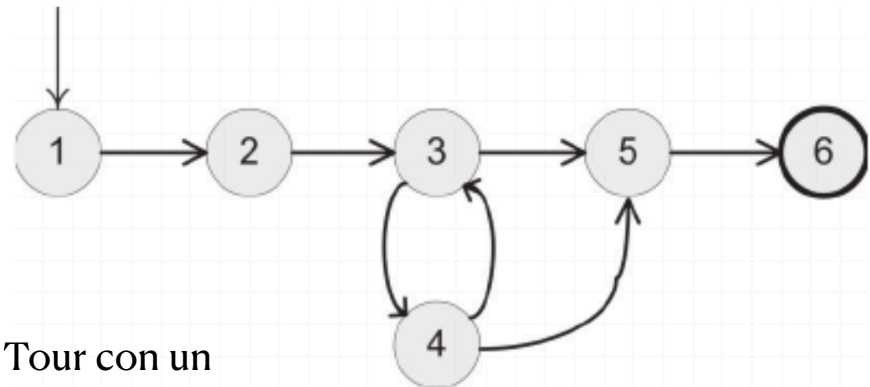
# Ejemplo



Tour del camino  
primario [1, 2, 3, 5, 6]  
sin sidetrips ni detours



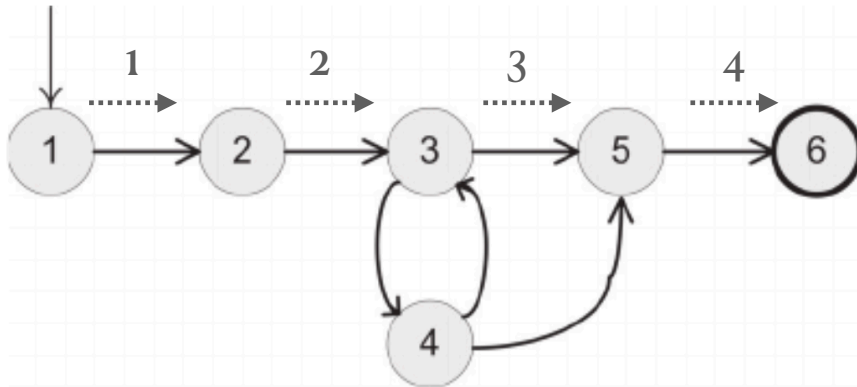
Tour con un  
sidetrip



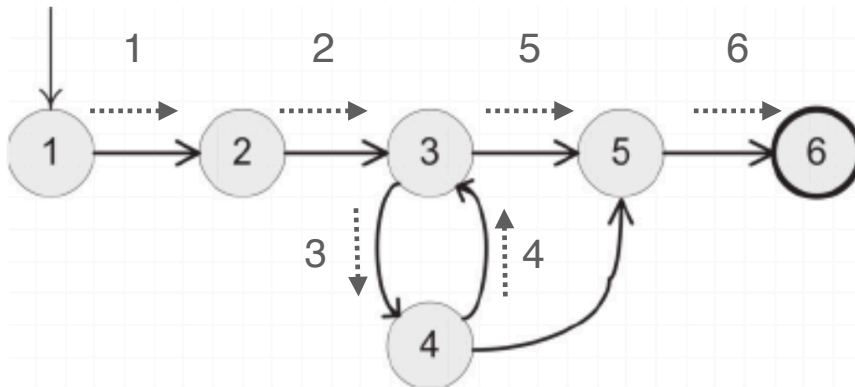
Tour con un  
detour



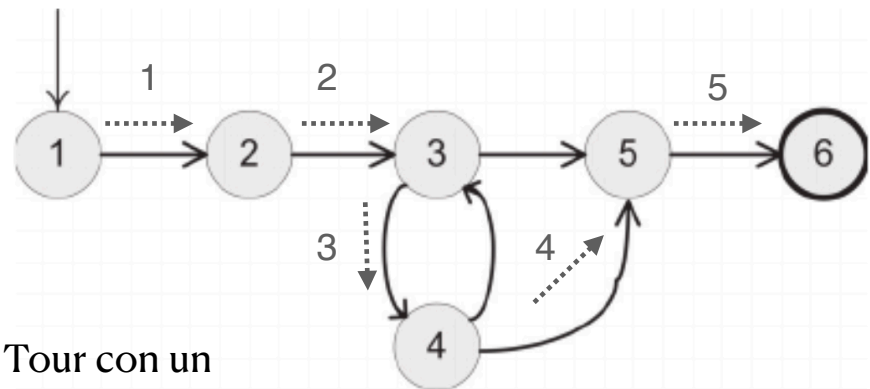
# Ejemplo



Tour del camino  
primario [1, 2, 3, 5, 6]  
sin sidetrips ni detours



Tour con un  
sidetrip



Tour con un  
detour

# Requisitos de test Inviabiles

Si **no** se permiten los **viajes adicionales** entonces los criterios estructurales suelen tener **más requisitos de test inviables**.

# Requisitos de test Inviabiles

Si **no** se permiten los **viajes adicionales** entonces los criterios estructurales suelen tener **más requisitos de test inviables**.

Por otro lado, si **permitimos** siempre los **viajes adicionales** entonces se **debilitan** los criterios de test.

# Requisitos de test Inviabiles

Si **no** se permiten los **viajes adicionales** entonces los criterios estructurales suelen tener **más requisitos de test inviables**.

Por otro lado, si **permitimos** siempre los **viajes adicionales** entonces se **debilitan** los criterios de test.

Por tanto, un compromiso entre estas situaciones es (**Best Effort Touring**):

# Requisitos de test Inviabiles

Si **no** se permiten los **viajes adicionales** entonces los criterios estructurales suelen tener **más requisitos de test inviables**.

Por otro lado, si **permitimos** siempre los **viajes adicionales** entonces se **debilitan** los criterios de test.

Por tanto, un compromiso entre estas situaciones es (**Best Effort Touring**):

- ★ Satisfacer tantos requisitos de test como sea posible sin usar viajes adicionales

# Requisitos de test Inviabiles

Si **no** se permiten los **viajes adicionales** entonces los criterios estructurales suelen tener **más requisitos de test inviables**.

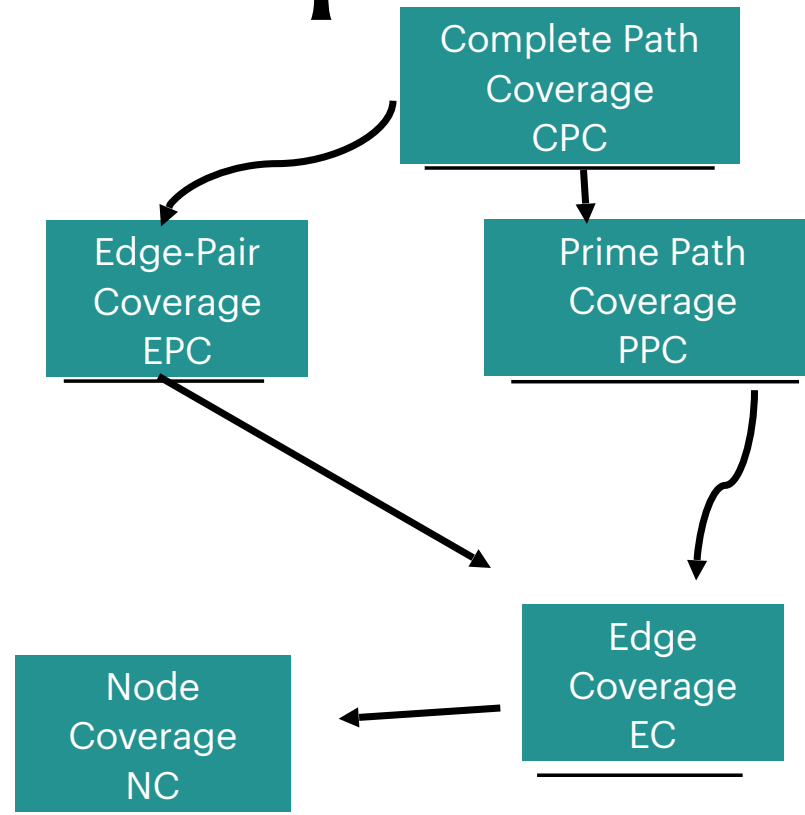
Por otro lado, si **permitimos** siempre los **viajes adicionales** entonces se **debilitan** los criterios de test.

Por tanto, un compromiso entre estas situaciones es (**Best Effort Touring**):

- ★ Satisfacer tantos requisitos de test como sea posible sin usar viajes adicionales
- ★ Permitir viajes adicionales para el resto de requisitos.

# Criterios de cobertura de grafos: subsumption

# Criterios de cobertura de grafos: subsumption





# Conclusiones

Los grafos son una abstracción muy potente para diseñar tests.

# Conclusiones

Los grafos son una abstracción muy potente para diseñar tests.

Los distintos criterios permiten compromisos entre costo y beneficio.

# Conclusiones

Los grafos son una abstracción muy potente para diseñar tests.

Los distintos criterios permiten compromisos entre costo y beneficio.

Hasta ahora solo hemos visto los grafos en el nivel de abstracción de diseño.

# Conclusiones

Los grafos son una abstracción muy potente para diseñar tests.

Los distintos criterios permiten compromisos entre costo y beneficio.

Hasta ahora solo hemos visto los grafos en el nivel de abstracción de diseño.

Los grafos aparecen en otras muchas situaciones a la hora de desarrollar software.

# Lectura recomendada

Paul Ammann, Jeff Offutt, Introduction to Software Testing, Cap 7.2.1 y 7.2.2